

docs.google.com – 全画面表示を終了するには、Esc を押します

# TL1

## 01\_08.カスタムプロパティ

日本工学院専門学校  
デザインカレッジ

# 今回の目標

日本工学院専門学校  
デザインカレッジ

- カスタムプロパティの使い方を習得する
- オブジェクトに、モデルのファイル名を関連付ける

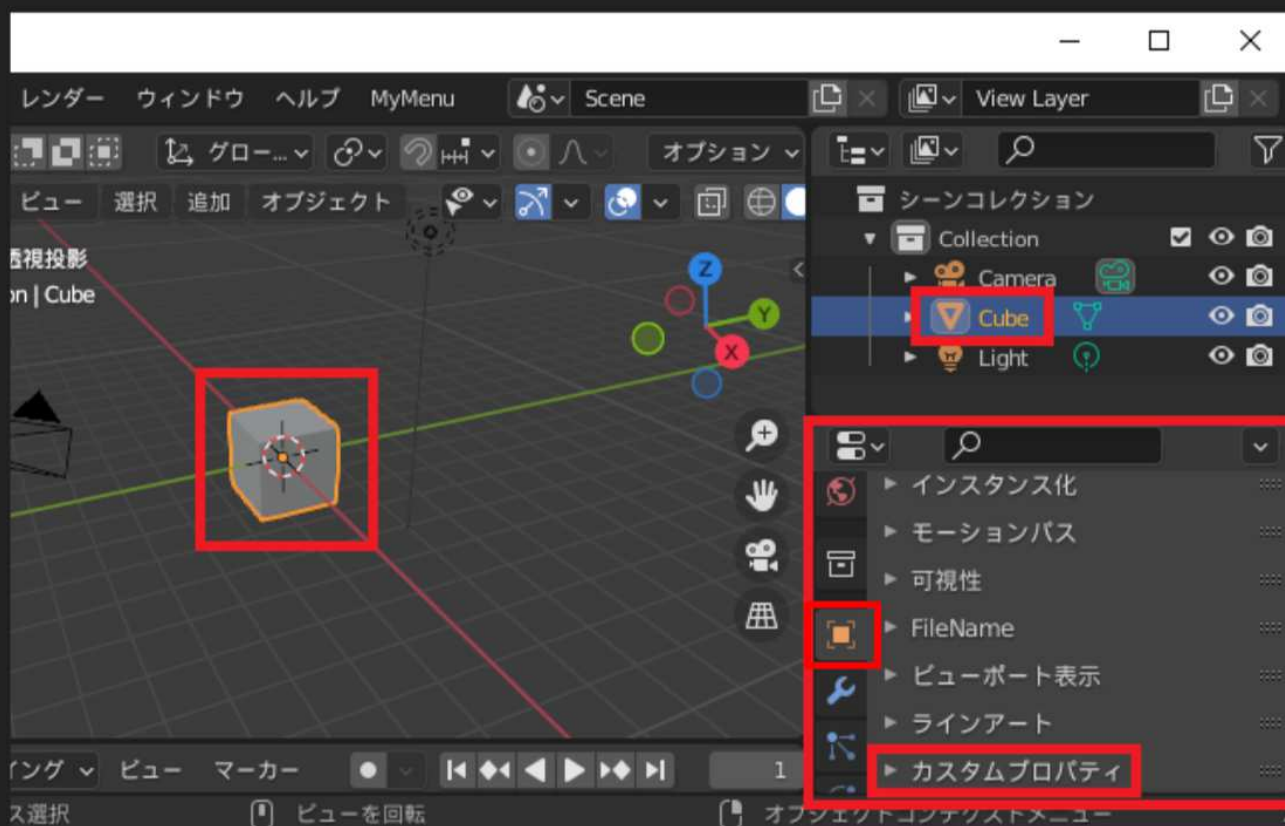
---

# カスタムプロパティ

---

# カスタムプロパティ

日本工学院専門学校  
デザインカレッジ



3Dビューポートかアウトライナーから何かオブジェクトを選択し、

プロパティウィンドウを見ると一番下に「カスタムプロパティ」という項目がある。

オブジェクトの持つプロパティ（メンバ変数と思ってよい）はあらかじめ決まっているが、カスタムプロパティはオブジェクトごとに自由に付け足すことが出来る。

# カスタムプロパティの追加

日本工学院専門学校  
デザインカレッジ



「追加」ボタンでプロパティが1つ増える。

「編集」ボタンで編集、「-」ボタンで項目を削除できる。

# カスタムプロパティの編集

日本工学院専門学校  
デザインカレッジ

プロパティの編集

プロパティ名

プロパティ値

デフォルト値

最小

最大

☐ ライブラリオーバーライドを...

☐ ソフトリミットを使用

ソフト最小

最大

ツールチップ

OK

編集ウィンドウで、

- プロパティの名前
- プロパティの値

などを編集できる。

値には文字列を設定することもできる。

ここで設定した値は、  
Blenderのデータを保存すればファイルを  
開き直してもちゃんと引き継がれる。

---

**Panel**

---

# Panel

日本工学院専門学校  
デザインカレッジ



ここまでで Operator、Menu を扱ったが、  
さらに別の組み込みクラス「Panel」を扱う。

Panelを使うことで、  
オブジェクトのプロパティウィンドウの項目を  
スクリプトで制御したり、UIを整えたりする。

ファイル名設定くらいであればカスタムプロパティだけで  
十分だが、ツールは第三者に提供するのが前提なので  
操作を分かりやすくするのも重要。



# Panelクラス

日本工学院専門学校  
デザインカレッジ

```
level_editor.py -class OBJECT_PT_file_name
```

```
#パネル ファイル名
class OBJECT_PT_file_name(bpy.types.Panel):
    """オブジェクトのファイルネームパネル"""
    bl_idname = "OBJECT_PT_file_name"
    bl_label = "FileName"
    bl_space_type = "PROPERTIES"
    bl_region_type = "WINDOW"
    bl_context = "object"

    # サブメニューの描画
    def draw(self, context):

        #パネルに項目を追加
        self.layout.label(text="Hello")
        self.layout.separator()
        self.layout.label(text="Hello2", icon="MESH_CUBE")
```

bpy.types.Panel を継承して  
新たなクラスを定義する。

書式はMenuにかなり近い。

bl\_label がプロパティウィンドウ  
での項目名になる。

bl\_context を "object" に  
設定することでオブジェクトを選択し  
た時用のパネル拡張になる。

draw() で項目の表示を制御する。

self.layout はUILayout型。  
カスタマイズの幅はかなり広い。  
([参考サイト](#))

# クラスリストに登録

日本工学院専門学校  
デザインカレッジ

```
# Blenderに登録するクラスリスト
classes = (
    MYADDON_OT_stretch_vertex,
    MYADDON_OT_create_ico_sphere,
    MYADDON_OT_export_scene,
    TOPBAR_MT_my_menu,
    OBJECT_PT_file_name,
)
```

`register()` で登録するクラスリストに追加しておけば、自動的にパネル拡張が適用される。

そういう意味では Menu より導入が楽。

# 実行結果

日本工学院専門学校  
デザインカレッジ



オブジェクトのプロパティウィンドウに  
`bl_label` に設定した名前の項目が増えた。

項目内は `draw()` に書いたレイアウトで表示される。

1行に複数の表示を詰め込んだりもできる。  
つまりUnityみたいに X, Y, Z を1行に並べたりできる。

# Operatorの登録

日本工学院専門学校  
デザインカレッジ

```
# サブメニューの描画  
def draw(self, context):
```

```
level_editor.py -class OBJECT_PT_file_name
```

```
#パネルに項目を追加
```

変更

```
self.layout.operator(MYADDON_OT_stretch_vertex.bl_idname, text=MYADDON_OT_stretch_vertex.bl_label)  
self.layout.operator(MYADDON_OT_create_ico_sphere.bl_idname, text=MYADDON_OT_create_ico_sphere.bl_label)  
self.layout.operator(MYADDON_OT_export_scene.bl_idname, text=MYADDON_OT_export_scene.bl_label)
```

▼ FileName

頂点を伸ばす

ICO球生成

シーン出力

Panel に Operator を登録することもできる。

MyMenu に登録していた Operator を試しに Panel に登録してみたらこうなった。

---

## カスタムプロパティを追加するOperator

---



# カスタムプロパティ追加Operator

日本工学院専門学校  
デザインカレッジ

```
level_editor.py -class MYADDON_OT_add_filename
```

```
#オペレータ カスタムプロパティ['file_name']追加
class MYADDON_OT_add_filename(bpy.types.Operator):
    bl_idname = "myaddon.myaddon_ot_add_filename"
    bl_label = "FileName 追加"
    bl_description = "['file_name']カスタムプロパティを追加します"
    bl_options = {"REGISTER", "UNDO"}

    def execute(self, context):

        #['file_name']カスタムプロパティを追加
        context.object["file_name"] = ""

        return {"FINISHED"}
```

分かりやすさの為に、  
新たなカスタムプロパティ  
「file\_name」を追加するボタン  
を作る。

つまり新たな Operator を作って、  
先ほどの Panel に登録する。

context は「今選択中の」という  
意味で捉えればよい。

context.object は今選択中のオブ  
ジェクト。

今選択中のオブジェクトに'file\_name'というカスタムプロパティを追加。  
カスタムプロパティは dictionary（連想配列。C++でいう std::map）として利用できる。

# クラスリストに登録

日本工学院専門学校  
デザインカレッジ

```
# Blenderに登録するクラスリスト
classes = (
    MYADDON_OT_stretch_vertex,
    MYADDON_OT_create_ico_sphere,
    MYADDON_OT_export_scene,
    TOPBAR_MT_my_menu,
    MYADDON_OT_add_filename,
    OBJECT_PT_file_name,
)
```

**register()** で登録するクラスリストに  
オペレータを追加しておく。

**登録順は特に関係ないので、  
自分で整理しなおしてもよい。**

# Operatorの登録

日本工学院専門学校  
デザインカレッジ

```
# サブメニューの描画  
def draw(self, context):
```

```
level_editor.py -class OBJECT_PT_file_name
```

変更

```
#パネルに項目を追加  
if "file_name" in context.object:  
    #既にプロパティがあれば、プロパティを表示  
    self.layout.prop(context.object, '["file_name"]', text=self.bl_label)  
else:  
    #プロパティがなければ、プロパティ追加ボタンを表示  
    self.layout.operator(MYADDON_OT_add_filename.bl_idname)
```

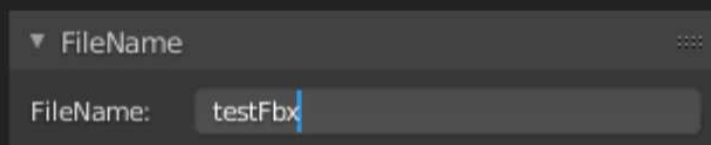
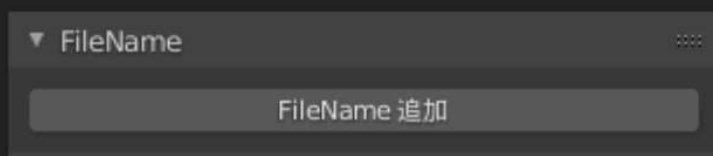
Panel に Operator を登録。

ただしカスタムプロパティ'file\_name'が既に追加済みの場合は、そのプロパティをUI項目として表示する。



# 実行結果

日本工学院専門学校  
デザインカレッジ



カスタムプロパティ '`file_name`' がない時は追加ボタンが表示され、

追加ボタンを押すとプロパティが追加されるので、プロパティ表示に切り替わる。

UI上から直接ファイル名を書き換えられる。  
この項目 '`file_name`' は、ゲーム側でこのオブジェクトに関連付けるモデルファイル名を指定する項目。

Blender上での表示モデルに合わせることも、異なるモデルを指定することもできる。

---

## 出力の整形

---

# 出力の整形

日本工学院専門学校  
デザインカレッジ

カスタムプロパティ 'file\_name' がエクスポートからの出力に対応していないので、その対応と、ローダーでの読み取りやすさを意識して既存の項目も少し整形してみる。

# 出力の整形

日本工学院専門学校  
デザインカレッジ

変更

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def parse_scene_recursive(self, file, object, level):  
  
    ...省略...  
    self.write_and_print(file, indent + object.type)  
    ...省略...  
    #トランスフォーム情報を表示  
    self.write_and_print(file, indent + "T %f %f %f" % (trans.x, trans.y, trans.z) )  
    self.write_and_print(file, indent + "R %f %f %f" % (rot.x, rot.y, rot.z) )  
    self.write_and_print(file, indent + "S %f %f %f" % (scale.x, scale.y, scale.z) )  
    #カスタムプロパティ 'file_name'  
    if "file_name" in object:  
        self.write_and_print(file, indent + "N %s" % object["file_name"])  
    self.write_and_print(file, indent + 'END')  
    self.write_and_print(file, '')  
  
    #子ノードへ進む(深さが1上がる)  
    for child in object.children:  
        self.parse_scene_recursive(file, child, level + 1)
```

# 実行結果

日本工学院専門学校  
デザインカレッジ

## 出力したファイル

```
1 SCENE
2
3 LIGHT
4 T 4.076245 1.005454 5.903862
5 R 37.261044 3.163707 106.936310
6 S 1.000000 1.000000 1.000000
7 END
8
9 CAMERA
10 T 7.358891 -6.925791 4.958309
11 R 63.559303 -0.000003 46.691948
12 S 1.000000 1.000000 1.000000
13 END
14
15 MESH
16 T 0.000000 0.000000 0.000000
17 R 0.000000 -0.000000 0.000000
18 S 1.000000 1.000000 1.000000
19 N testFbx
20 END
21
22 MESH
23 T 3.000000 3.000000 3.000000
24 R 0.000000 -0.000000 0.000000
25 S 1.000000 1.000000 1.000000
26 N testFbx
27 END
```

```
28
29 EMPTY
30 T -3.000000 -3.000000 0.000000
31 R 0.000000 -0.000000 0.000000
32 S 1.000000 1.000000 1.000000
33 END
34
35 MESH
36 T 5.000000 0.000000 0.000000
37 R 45.000000 -0.000000 0.000000
38 S 2.000000 2.000000 2.000000
39 N ico_ball
40 END
41
42 MESH
43 T 0.000000 0.000000 0.000000
44 R 0.000000 -0.000000 0.000000
45 S 1.000000 1.000000 1.000000
46 N ico_ball
47 END
```

- 半角スペース区切りでの読み取り
- オブジェクトの区切りを明確にする

あたりを意識してみた。

オブジェクト名はゲーム側で使わない可能性が高いので削除した。  
ただし残しておくと、ローダーのデバッグには役立つだろう。

# まとめ

カスタムプロパティとPanelについて学んだ。

これらを併用することで、  
Blender側で自分の好きな項目を作り、ゲーム側にデータとして  
読み込ませることができる。

（もちろんエクスポーターとローダーも改造が必要）

今回はレベルエディタとして絶対に必要であろう「モデルファイル名」を  
オブジェクトごとに指定できるようにしてみた。



# TL1

01\_08.カスタムプロパティ

日本工学院専門学校  
デザインカレッジ